

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
Факультет Программной инженерии и компьютерной техники

Системы искусственного интеллекта
Лабораторная работа №4

Выполнил студент
Егорова Варвара Александровна
Группа Р3323
Преподаватель: Королева Юлия Александровна

Содержание	
Текст задания.....	3
Реализация.....	4
Оценка модели.....	11
Вывод.....	12
Приложение.....	13

Текст задания

Реализовать класс моделей Decision Tree, Random Forest и Gradient Boosting, обучить их на задачи классификации и регрессии на заданных датасетах (cifar10 и london-bike-sharing-dataset соответственно). Провести их сравнительный анализ.

Реализация

Далее на листингах 1-3 представлена реализация класса моделей Decision Tree, Random Forest и Gradient Boosting.

Листинг 1 - Реализация модели Decision Tree.

```
import torch

class DecisionTree:
    def __init__(self, max_depth=10, min_samples_split=2,
classification=True):
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.classification = classification
        self.tree = None

    def _gini(self, y):
        _, counts = torch.unique(y, return_counts=True)
        p = counts.float() / len(y)
        return 1 - torch.sum(p*p)

    def _variance(self, y):
        return torch.var(y.float())

    def _best_split(self, X, y):
        best_feat, best_thr = None, None
        best_gain = -1e18

        m, n = X.shape

        for feat in range(n):
            col = X[:, feat]
            min_v = torch.min(col)
            max_v = torch.max(col)
            if min_v == max_v:
                continue
            thresholds = torch.linspace(min_v, max_v, steps=16)

            for thr in thresholds:
                left_mask = col <= thr
                right_mask = ~left_mask
                if left_mask.sum() == 0 or right_mask.sum() == 0:
                    continue

                left_y = y[left_mask]
                right_y = y[right_mask]
```

```

        if self.classification:
            impurity_parent = self._gini(y)
            gain = impurity_parent - (
                len(left_y)/len(y) * self._gini(left_y) +
                len(right_y)/len(y) * self._gini(right_y)
            )
        else:
            var_parent = self._variance(y)
            gain = var_parent - (
                len(left_y)/len(y) * self._variance(left_y) +
                len(right_y)/len(y) * self._variance(right_y)
            )

        if gain > best_gain:
            best_gain = gain
            best_feat = feat
            best_thr = float(thr)

    return best_feat, best_thr

def _build(self, X, y, depth):
    if depth >= self.max_depth or len(y) < self.min_samples_split:
        if self.classification:
            classes, counts = torch.unique(y, return_counts=True)
            return int(classes[torch.argmax(counts)])
        else:
            return float(torch.mean(y.float()))

    feat, thr = self._best_split(X, y)
    if feat is None:
        if self.classification:
            classes, counts = torch.unique(y, return_counts=True)
            return int(classes[torch.argmax(counts)])
        else:
            return float(torch.mean(y.float()))

    left_mask = X[:, feat] <= thr
    right_mask = ~left_mask

    return {
        "feature": feat,
        "threshold": thr,
        "left": self._build(X[left_mask], y[left_mask], depth + 1),
        "right": self._build(X[right_mask], y[right_mask], depth +
1),
    }

```

```

def fit(self, X, y):
    self.X = torch.as_tensor(X, dtype=torch.float32)
    y = torch.as_tensor(y, dtype=torch.int64 if self.classification
else torch.float32)
    self.tree = self._build(self.X, y, 0)

def _predict_one(self, node, x):
    if not isinstance(node, dict):
        return node
    if x[node["feature"]] <= node["threshold"]:
        return self._predict_one(node["left"], x)
    else:
        return self._predict_one(node["right"], x)

def predict(self, X):
    X = torch.as_tensor(X, dtype=torch.float32)
    return torch.tensor([self._predict_one(self.tree, x) for x in
X])

```

Листинг 2 - Реализация модели Random Forest.

```

import torch
import random

class RandomForest:
    def __init__(self, n_estimators=10, max_depth=10,
min_samples_split=2,
                max_features=None, classification=True):
        self.n_estimators = n_estimators
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.max_features = max_features
        self.classification = classification
        self.trees = []

    def _bootstrap(self, X, y):
        n = len(X)
        idxs = torch.randint(0, n, (n,))
        return X[idxs], y[idxs]

    def _subset_features(self, X):
        n_features = X.shape[1]
        if self.max_features is None:
            k = int(n_features ** 0.5) if self.classification else
int(n_features * 0.7)

```

```

else:
    k = self.max_features
    idxs = torch.randperm(n_features)[:k]
    return idxs

def fit(self, X, y):
    X = torch.tensor(X, dtype=torch.float32)
    y = torch.tensor(y, dtype=torch.int64 if self.classification
else torch.float32)

    for i in range(self.n_estimators):
        print(f" Tree {i+1}/{self.n_estimators}")
        Xs, ys = self._bootstrap(X, y)
        feat_idx = self._subset_features(X)
        Xt = Xs[:, feat_idx]

        tree = DecisionTree(
            max_depth=self.max_depth,
            min_samples_split=self.min_samples_split,
            classification=self.classification
        )
        tree.fit(Xt, ys)
        self.trees.append((tree, feat_idx))

def predict(self, X):
    X = torch.tensor(X, dtype=torch.float32)
    preds = []

    for tree, feat_idx in self.trees:
        Xt = X[:, feat_idx]
        preds.append(tree.predict(Xt))

    preds = torch.stack(preds, dim=0)

    if self.classification:
        out = []
        for i in range(preds.shape[1]):
            values, counts = torch.unique(preds[:, i],
return_counts=True)
            out.append(int(values[torch.argmax(counts)]))
        return torch.tensor(out)
    else:
        return torch.mean(preds.float(), dim=0)

```

Листинг 3 - Реализация модели Gradient Boosting.

```
import torch

class GradientBoosting:
    def __init__(self, n_estimators=20, learning_rate=0.1, max_depth=3,
classification=True):
        self.n_estimators = n_estimators
        self.learning_rate = learning_rate
        self.max_depth = max_depth
        self.classification = classification
        self.models = []
        self.base = None
        self.num_classes = None
        self.device = torch.device("cpu")

    def fit(self, X, y):
        X = torch.as_tensor(X, dtype=torch.float32)
        self.device = X.device

        if self.classification:
            y = torch.as_tensor(y, dtype=torch.long, device=self.device)
            self.num_classes = int(y.max().item() + 1)

            counts = torch.bincount(y,
minlength=self.num_classes).float()
            p = counts / counts.sum()
            self.base = torch.log(p + 1e-9).to(self.device)

            pred = self.base.unsqueeze(0).repeat(len(X), 1)

            for i in range(self.n_estimators):
                print(f" Estimator {i+1}/{self.n_estimators}")
                prob = torch.softmax(pred, dim=1)
                y_one_hot = torch.nn.functional.one_hot(y,
num_classes=self.num_classes).float()

                residual = (y_one_hot - prob).to(self.device)
                trees = []

                for c in range(self.num_classes):
                    t = DecisionTree(max_depth=self.max_depth,
min_samples_split=2,
classification=False)
                    t.fit(X.cpu(), residual[:, c].cpu())
                    update = t.predict(X.cpu()).to(self.device)
```

```

        pred[:, c] += self.learning_rate * update
        trees.append(t)

    self.models.append(trees)

    else:
        y = torch.as_tensor(y, dtype=torch.float32,
device=self.device)
        self.base = y.mean().to(self.device)
        pred = torch.full_like(y, self.base)

        for i in range(self.n_estimators):
            print(f" Tree {i+1}/{self.n_estimators}")
            residual = (y - pred).to(self.device)

            t = DecisionTree(max_depth=self.max_depth,
                            min_samples_split=2,
                            classification=False)

            t.fit(X.cpu(), residual.cpu())
            update = t.predict(X.cpu()).to(self.device)

            pred += self.learning_rate * update
            self.models.append(t)

    def predict(self, X):
        X = torch.as_tensor(X, dtype=torch.float32)
        X_cpu = X.cpu()

        if self.classification:
            pred = self.base.unsqueeze(0).repeat(len(X),
1).to(self.device)

            for trees in self.models:
                for c, t in enumerate(trees):
                    pred[:, c] += self.learning_rate *
t.predict(X_cpu).to(self.device)

            return torch.argmax(pred, dim=1)

        else:
            pred = torch.full((len(X),), self.base, device=self.device)

            for t in self.models:
                pred += self.learning_rate *
t.predict(X_cpu).to(self.device)

```

```
return pred
```

Оценка модели

Далее на Листингах 4-5 представлен результат работы моделей на задаче регрессии и классификации.

Листинг 4 - Результат работы моделей на задаче регрессии.

	MSE	RMSE	MAE
Gradient Boosting	155229.312500	393.991513	259.642334
Decision Tree	253816.453125	503.801998	307.409454
Random Forest	370416.843750	608.618800	438.382416

Листинг 5 - Результат работы моделей на задаче классификации.

	accuracy	f1	precision
Decision Tree	0.273333	0.259025	0.271235
Gradient Boosting	0.328000	0.313188	0.330519
Random Forest	0.365333	0.351364	0.395835

По результатам эксперимента на задаче регрессии лучшей моделью стал Gradient Boosting за счет последовательного обучения деревьев, где каждое следующее исправляет ошибки предыдущего. Так же лучший результат он показал на классификации. Random forest оказался вторым на классификации и третьим на регрессии, что неожиданно. Вероятно, из-за небольшого количества деревьев и недостаточной их глубины. Decision Tree стал вторым на регрессии и третьим на классификации, что ожидаемо, поскольку одно дерево может хорошо классифицировать заданный датасет, но плохо экстраполировать его на тестовые данные.

Вывод

В ходе выполнения лабораторной работы я ознакомилась с решающими деревьями и их ансамблями. Были написаны классы моделей Decision Tree, Random Forest и Gradient Boosting, которые были обучены на реальных данных, и проведены эксперименты на сравнение качества этих моделей.

Приложение

Исходный код лабораторной работы представлен по ссылке:
<https://colab.research.google.com/drive/1Z0nzbicgpe8xaFN8gzYknee5nYqXjtz?usp=sharing>